TaskFlow Manager



Interface de gestion de projets

David Ongaro
03/11/2026

Sommaire

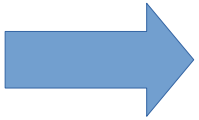
- Contexte
- Conception
- Back-end
- Front-end
- Qualité et déploiement
- Démo

Contexte

TaskFlow est une société qui gère ses projets et ses tâches en équipe

Fonctionnement initial

- Gestion par des requêtes via API REST (Bruno, Postman…)
- Accès technique restreint à des profils expérimentés



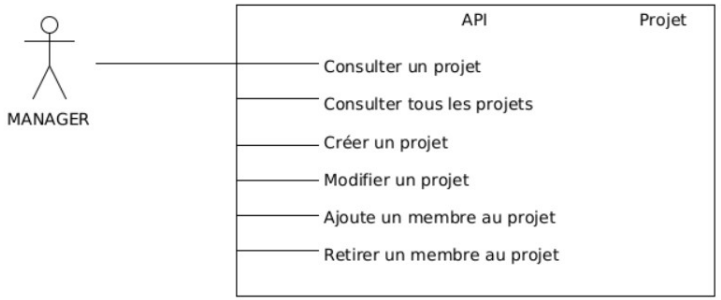
Besoin

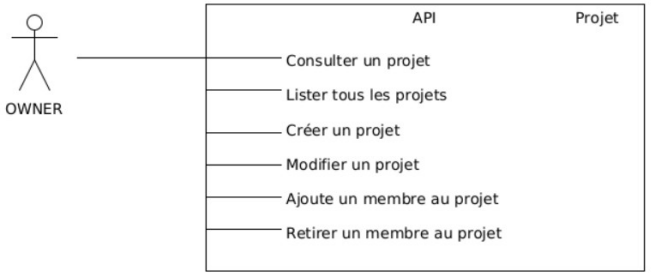
- Création d'interface web simple et accessible à tous
- Gestion facile des projets, tâches…
- Gestion des droits (manager, membre…)

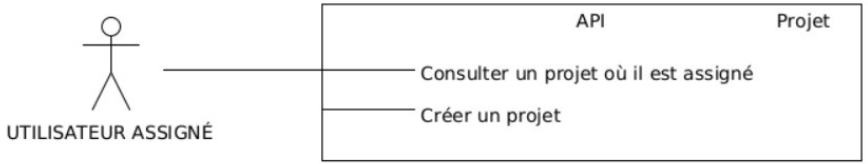
🎯 **Objectif :** Faciliter et accélérer la gestion des projets au quotidien

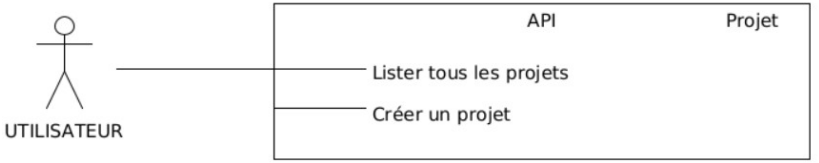
Conception

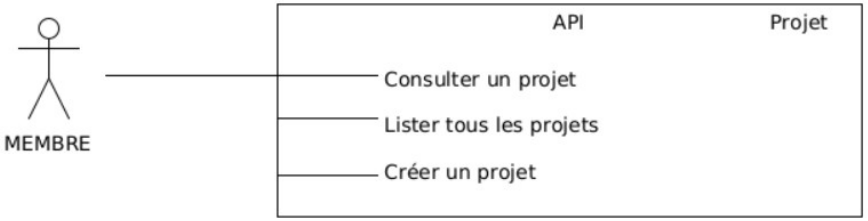
Rôles et cas d'utilisation :











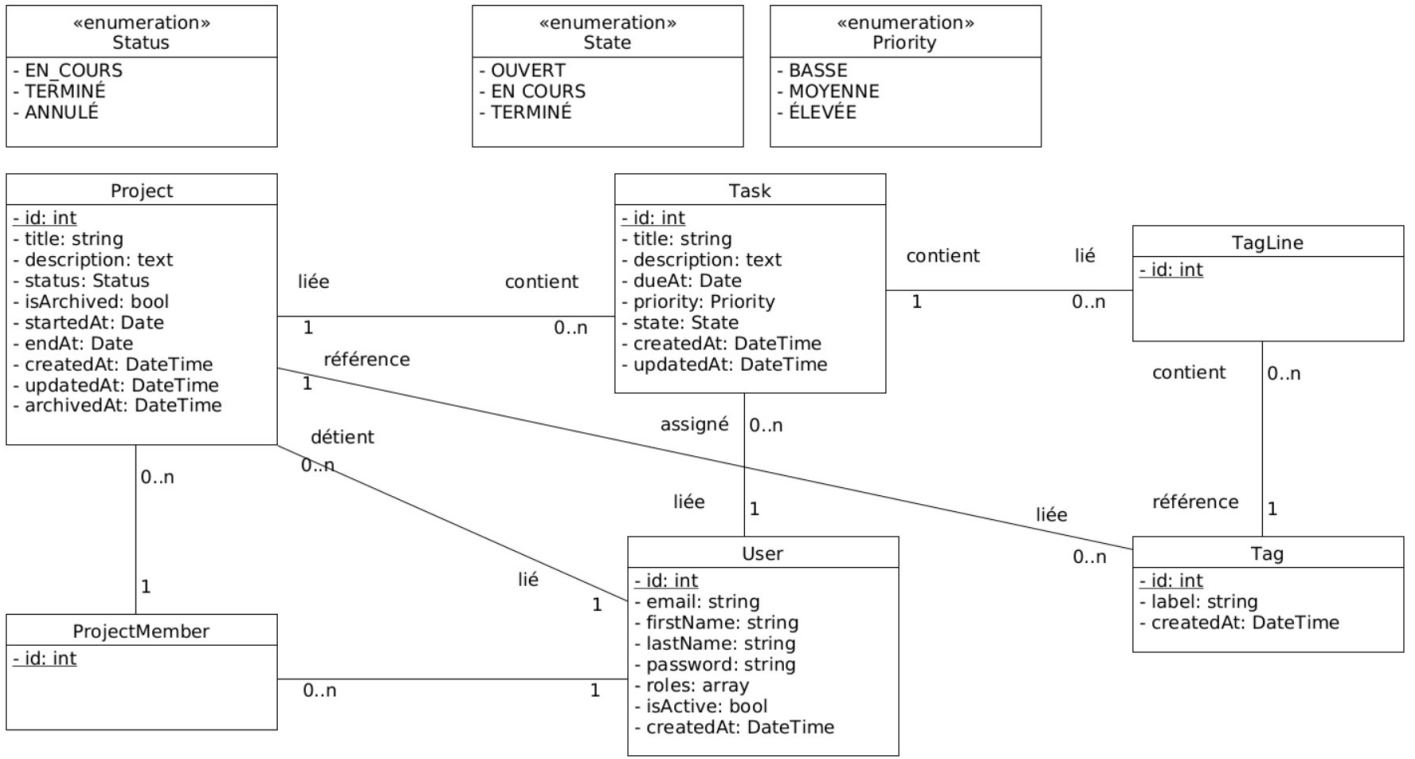
Conception

Droits d’accès :

Action	MANAGER	OWNER	MEMBRE	ASSIGNÉ TÂCHE	UTILISATEUR	HORS PROJET
Auth						
POST /auth/login						
GET /me						
PATCH /me						
Utilisateurs						
Lister / voir / créer / modifier /supprimer un user						
Projets (en cours)						
Créer un projet						
Lister les projets						
Voir un projet spécifique						
Modifier / archiver / restaurer						
Ajouter / retirer des membres						
Projets (archivés)						
Lecture (projet, tâches, tags)						
Écriture						
Tâches						
Lister / voir celles du projet						
Créer une tâche						
Modifier : titre, description, Priorité, échéance, assignee						
Supprimer une tâche						
Passer l’état à en cours						
Passer l’état à ouvert / terminé						
Associer un tag à une tâche						
Tags						
Lister les tags						
Créer / renommer / supprimer un tag						

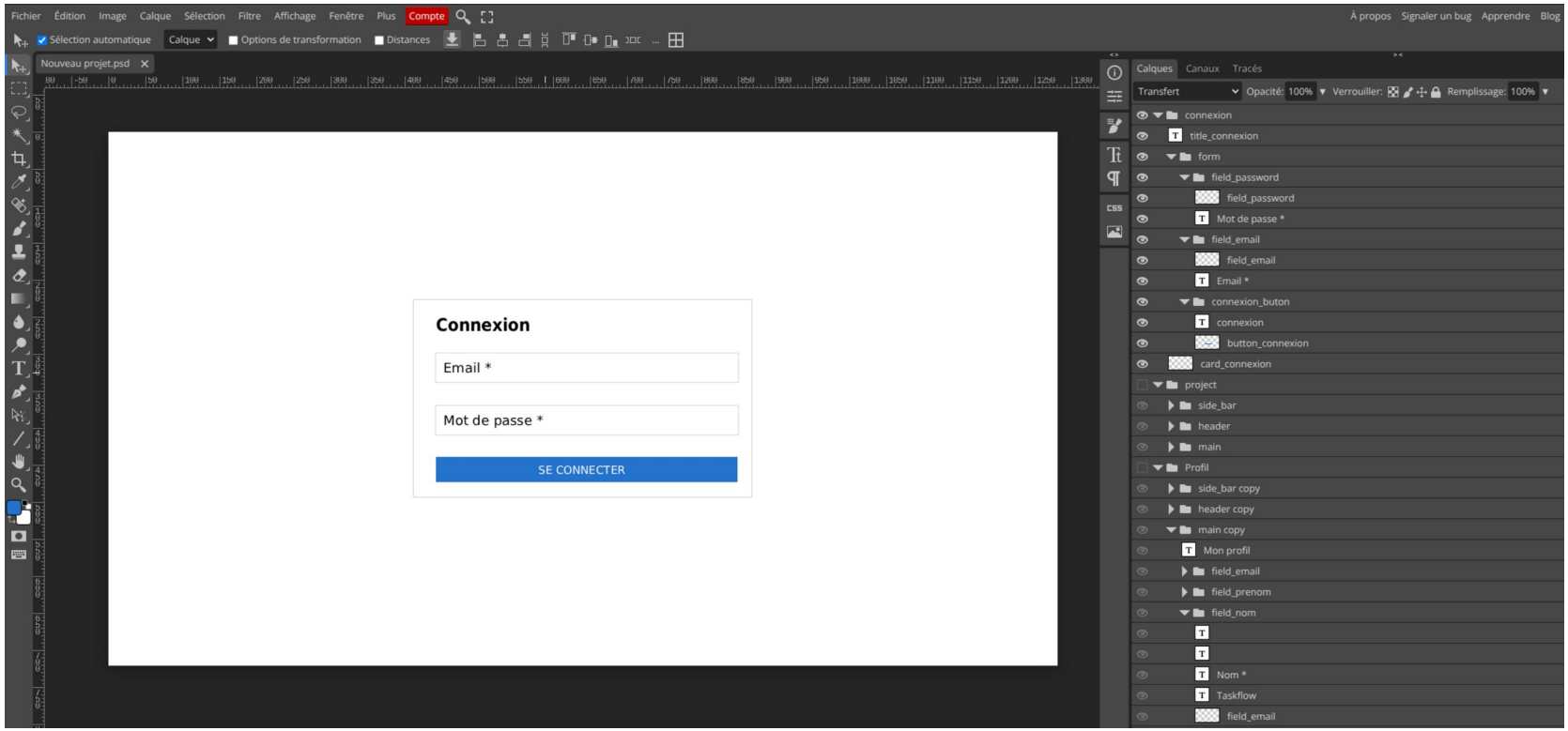
Conception

Modèle de données :

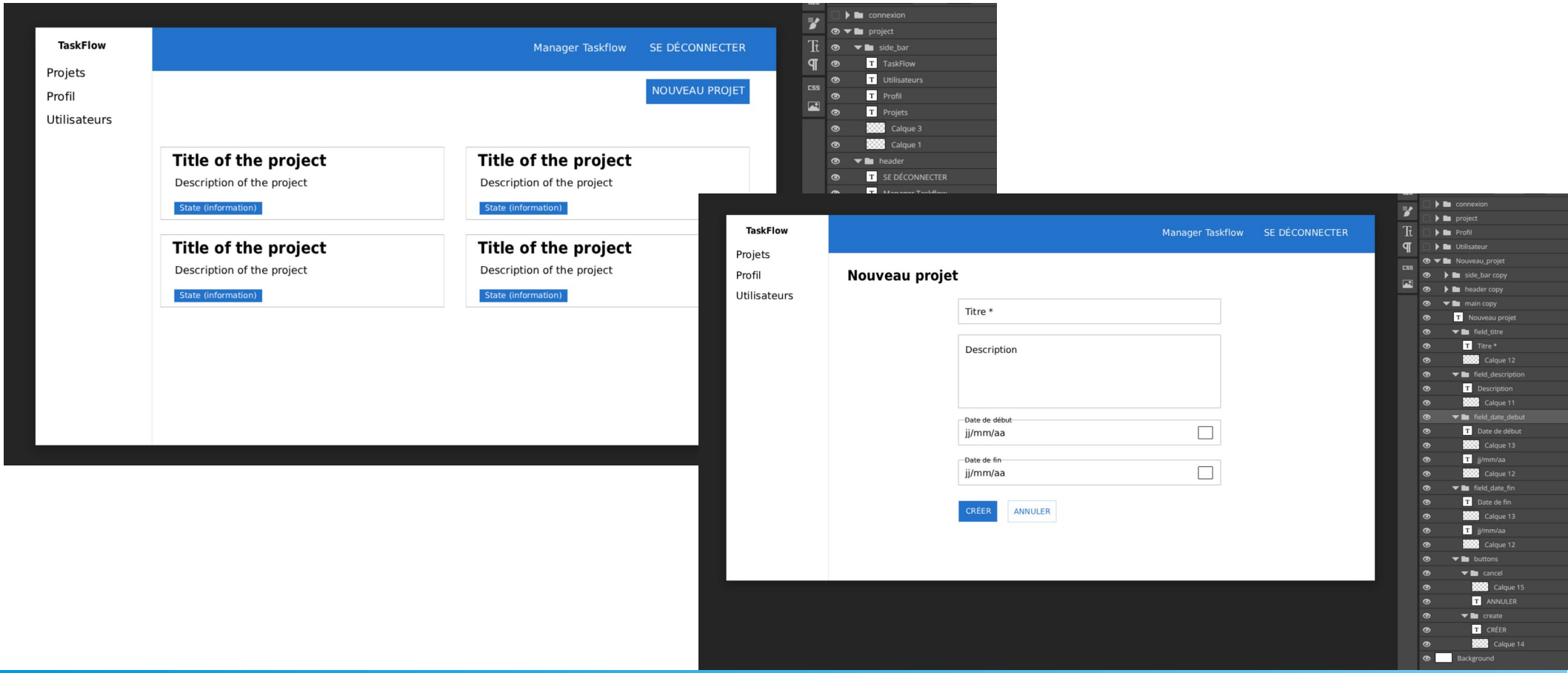


Conception

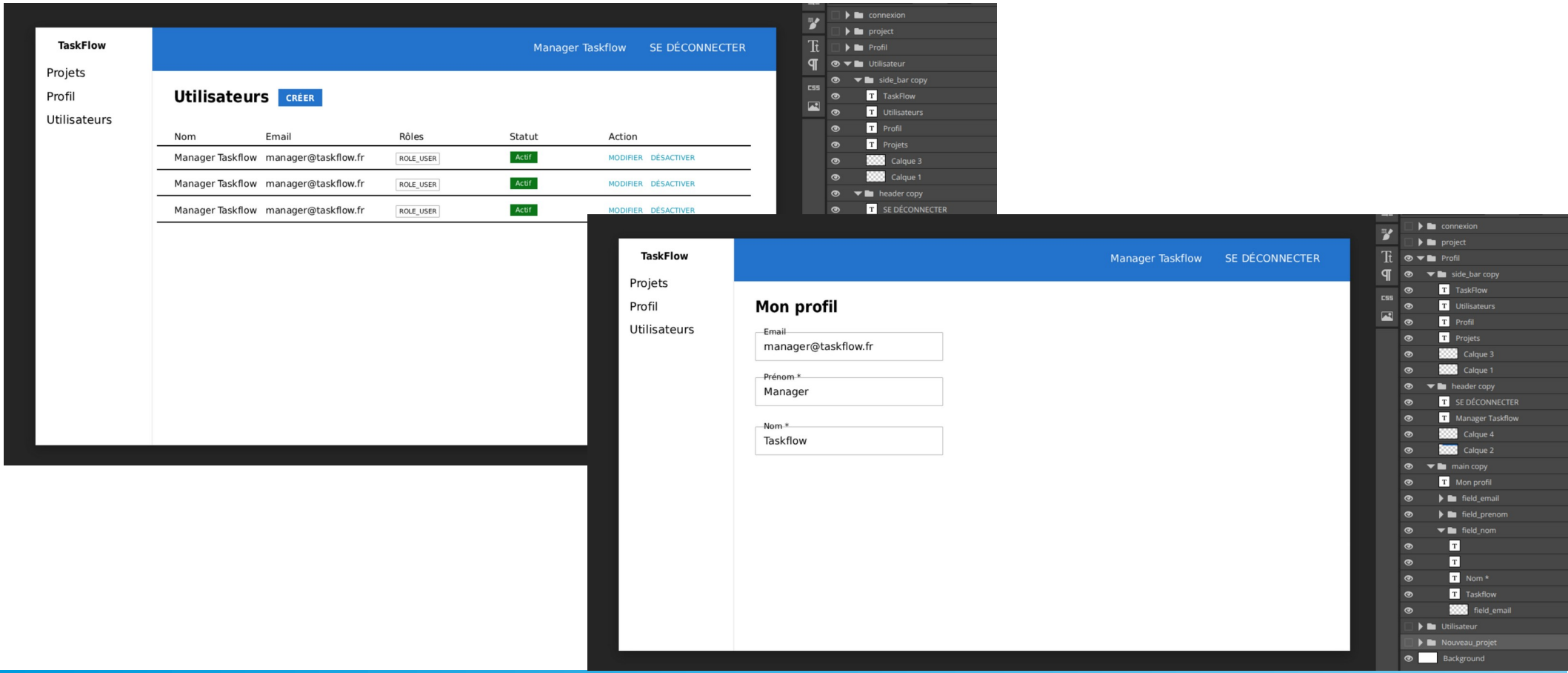
Maquettes :



Conception


TaskFlow
Projets
Profil
Utilisateurs
Manager Taskflow
SE DÉCONNECTER
NOUVEAU PROJET
Title of the project
Description of the project
State (information)
Nouveau projet
Titre *
Description
Date de début
jj/mm/aa
Date de fin
CRÉER
ANNULER

Conception


TaskFlow
Projets
Profil
Utilisateurs
Manager Taskflow
SE DÉCONNECTER
Utilisateurs
CRÉER
Nom
Email
Rôles
Statut
Action
Manager Taskflow
manager@taskflow.fr
ROLE_USER
Actif
MODIFIER
DÉSACTIVER
Manager Taskflow
manager@taskflow.fr
ROLE_USER
Actif
MODIFIER
DÉSACTIVER
Manager Taskflow
manager@taskflow.fr
ROLE_USER
Actif
MODIFIER
DÉSACTIVER
connexion
project
Profil
Utilisateur
side_bar copy
TaskFlow
Utilisateurs
Profil
Projets
Calque 3
Calque 1
header copy
SE DÉCONNECTER
TaskFlow
Projets
Profil
Utilisateurs
Manager Taskflow
SE DÉCONNECTER
Mon profil
Email
manager@taskflow.fr
Prénom *
Manager
Nom *
Taskflow
connexion
project
Profil
side_bar copy
TaskFlow
Utilisateurs
Profil
Projets
Calque 3
Calque 1
header copy
SE DÉCONNECTER
Manager Taskflow
Calque 4
Calque 2
main copy
Mon profil
field_email
field_prenom
field_nom
Nom *
Taskflow
field_email
Utilisateur
Nouveau_projet
Background

Back-end

Descriptif de l'API :

API REST

Symfony 7.2

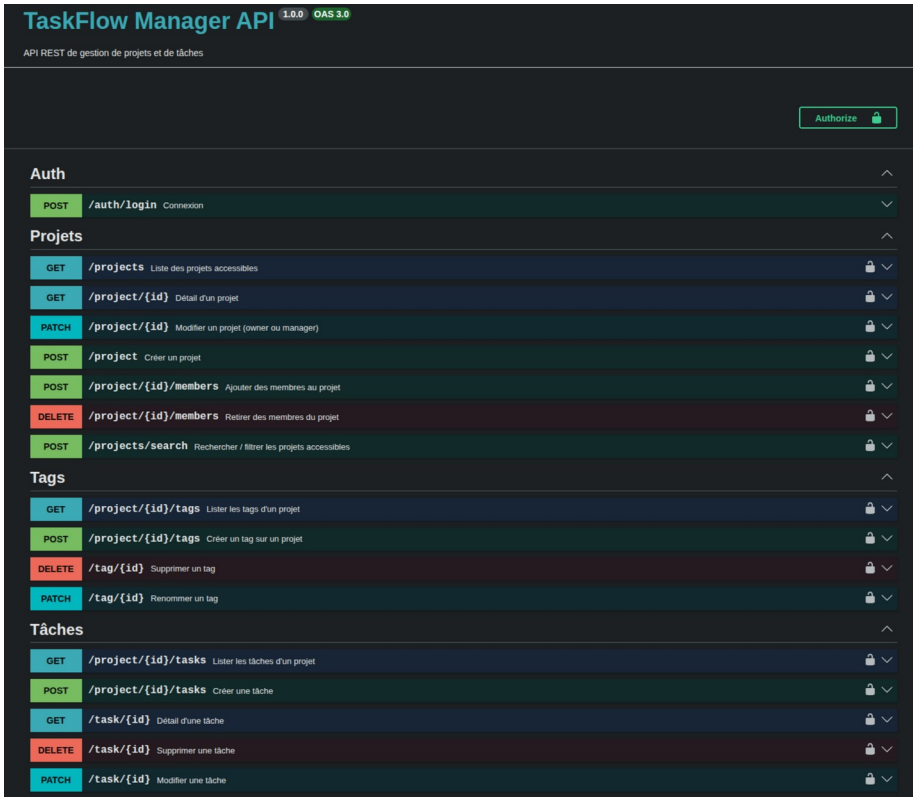
PHP 8.2

Base MySQL 8

5 ressources

Auth | Users | Projects | Tasks | Tags

Toutes les routes sont protégées par JWT, sauf le login et le Swagger



Back-end

Architecture en couches :



Envoie la demande via le navigateur

Reçoit la requête, vérifie les droits, renvoie du JSON

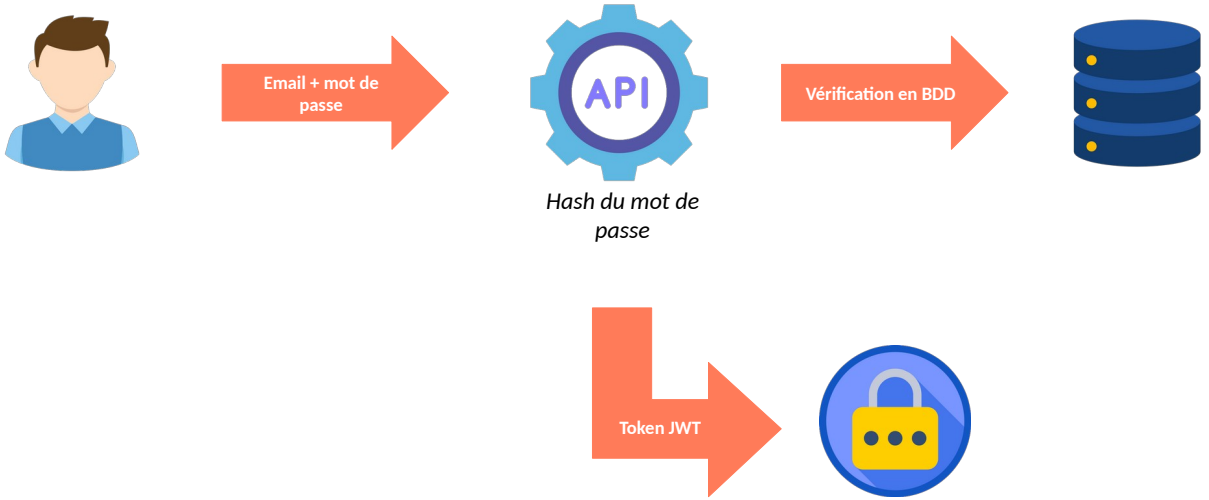
Valide et transfère les données entrantes

Requête Doctrine

Porte les règles métier

Base de données

Sécurité, authentification JWT :



Back-end

Sécurité, contrôle d’accès :



ROLE_MANAGER



ROLE_USER

- Chacun ses droits
- Des droits par projet
- Projet non modifiable (terminé/archivé)

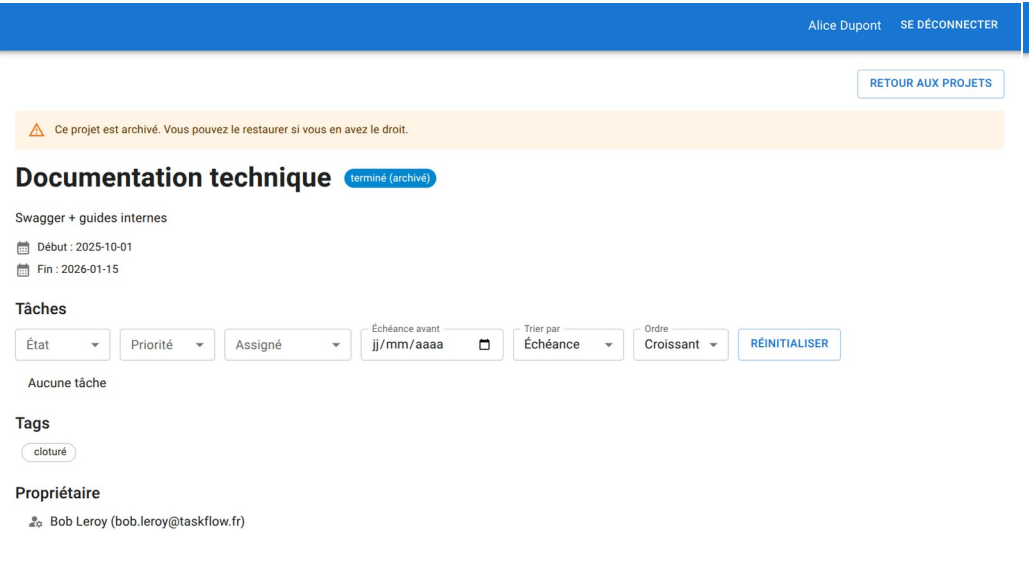
```
7 usages   David
public function canBeModifiedBy(User $user): bool
{
    if (!$this->isEditable()) {
        return false;
    }

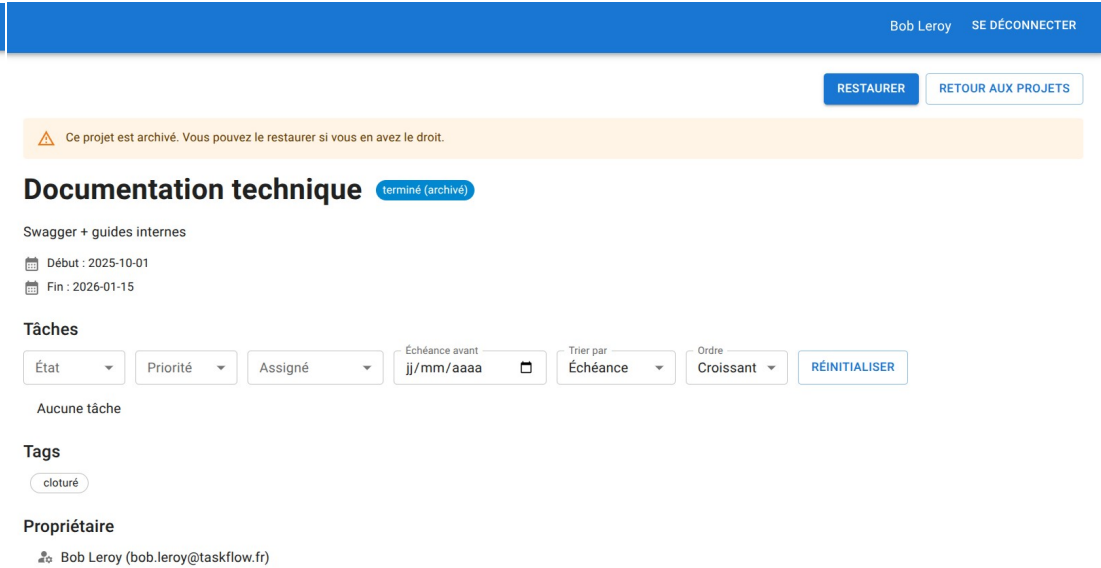
    if ($user->isManager()) {
        return true;
    }

    $owner = $this->owner;

    return $owner instanceof User && $owner->getId() === $user->getId();
}
```

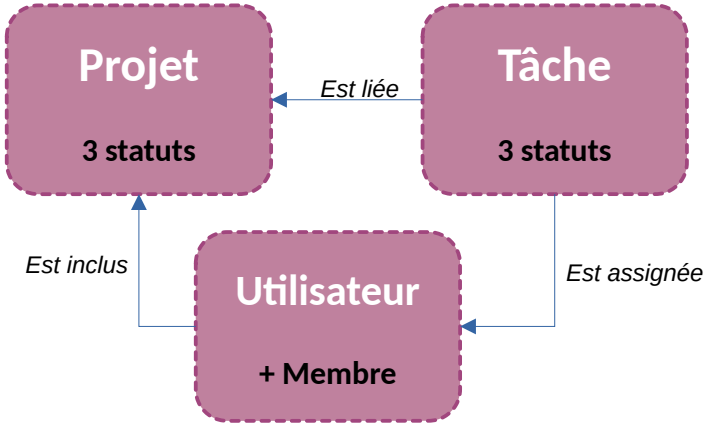
Back-end


Alice Dupont
SE DÉCONNECTER
RETOUR AUX PROJETS
Ce projet est archivé. Vous pouvez le restaurer si vous en avez le droit.
Documentation technique
terminé (archivé)
Swagger + guides internes
Début : 2025-10-01
Fin : 2026-01-15
Tâches
État
Priorité
Assigné
Échéance avant
jj/mm/aaaa
Trier par
Échéance
Ordre
Croissant
RÉINITIALISER
Aucune tâche
Tags
cloturé
Propriétaire
Bob Leroy (bob.leroy@taskflow.fr)


Bob Leroy
SE DÉCONNECTER
RESTAURER
RETOUR AUX PROJETS
Ce projet est archivé. Vous pouvez le restaurer si vous en avez le droit.
Documentation technique
terminé (archivé)
Swagger + guides internes
Début : 2025-10-01
Fin : 2026-01-15
Tâches
État
Priorité
Assigné
Échéance avant
jj/mm/aaaa
Trier par
Échéance
Ordre
Croissant
RÉINITIALISER
Aucune tâche
Tags
cloturé
Propriétaire
Bob Leroy (bob.leroy@taskflow.fr)

Back-end

Règles métier :



basse	ouvert	Écran de connexion	Claire Bernard	⋮
moyenne	en cours	Maquettes du projet	Manager Taskflow	⋮
élevée	terminé	Initialisation projet	Claire Bernard	⋮

ios mobile

Intégration SSO

Projet annulé faute de budget

annulé (archivé)

Application mobile TaskFlow

MVP iOS / Android

en cours

Migration base de données

MySQL 8 → nouvelle infra

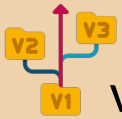
terminé (archivé)

Back-end

Accès aux données :

4 entités Doctrine

- Users
- Projects
- Tasks
- Tags



Migration versionnée

Fixtures de test

Paramètres liés

Protection injection SQL



```php
1 usage  David
public function searchByUser(User $user, SearchProjectRequest $criteria): array
{
    $query = $this->createQueryBuilder( alias: 'project');

    if (!$user->isManager()) {
        $query->where( ...predicates: 'project.owner = :user')
            ->orWhere( ...where: ':user MEMBER OF project.members')
            ->setParameter( key: 'user', $user);
    }

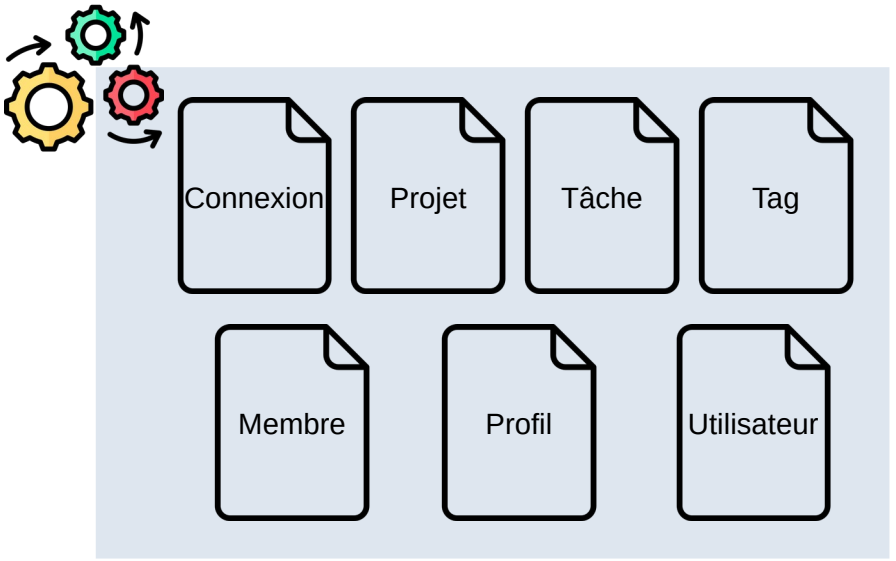
    if (!empty($criteria->status)) {
        $query->andWhere( ...where: 'project.status IN (:statuses)')
            ->setParameter( key: 'statuses', $criteria->status);
    }

    if (null !== $criteria->archived) {
        $query->andWhere( ...where: 'project.isArchived = :archived')
            ->setParameter( key: 'archived', $criteria->archived);
    }
```

Front-end

Descriptif :

React Vite MUI React Router



Le front est une SPA qui ne fait qu'afficher les données et appeler l'API.
Toute la logique métier et la sécurité sont côté serveur.

Front-end

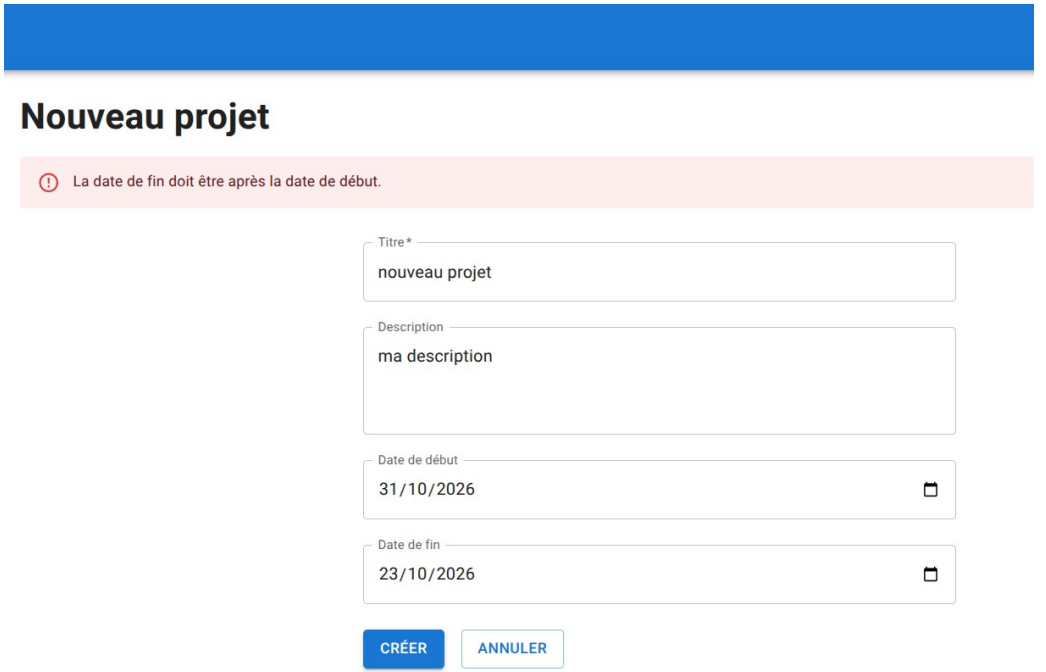
Architecture :

api/	pages/	components/	contexts/ + hooks/	utils/
Appels HTTP Un fichier par ressource (projects.js, tasks.js…) Un client centralisé (client.js)	Un écran par route (Projects, ProjectDetail, Users, Profile, Login…)	Composants réutilisables (formulaires, modales, filtres, layout)	État d'authentification partagé (AuthProvider, useAuth)	Fonctions pures (permissions.js, tasks.js, projects.js), faciles à tester

Front-end

Communication avec l’API :

- URL de base dans VITE_API_BASE_URL
- Ajout automatique du header Authorization: Bearer <token>
- En cas de 401, le token est supprimé et l'utilisateur est redirigé vers le login
- Les messages d'erreur renvoyés par l'API sont réutilisés pour l'affichage



Front-end

Authentification et routes protégées :

- AuthProvider (React Context) : garde l'utilisateur connecté. Il charge le profil avec /me
- Token stocké en sessionStorage : il est effacé à la fermeture de l'onglet.
- Routes imbriquées : ProtectedRoute (connecté), ManagerRoute (manager, pour /users)
- Affichage selon les droits avec permissions.js : canEditProject, canManageTasks…

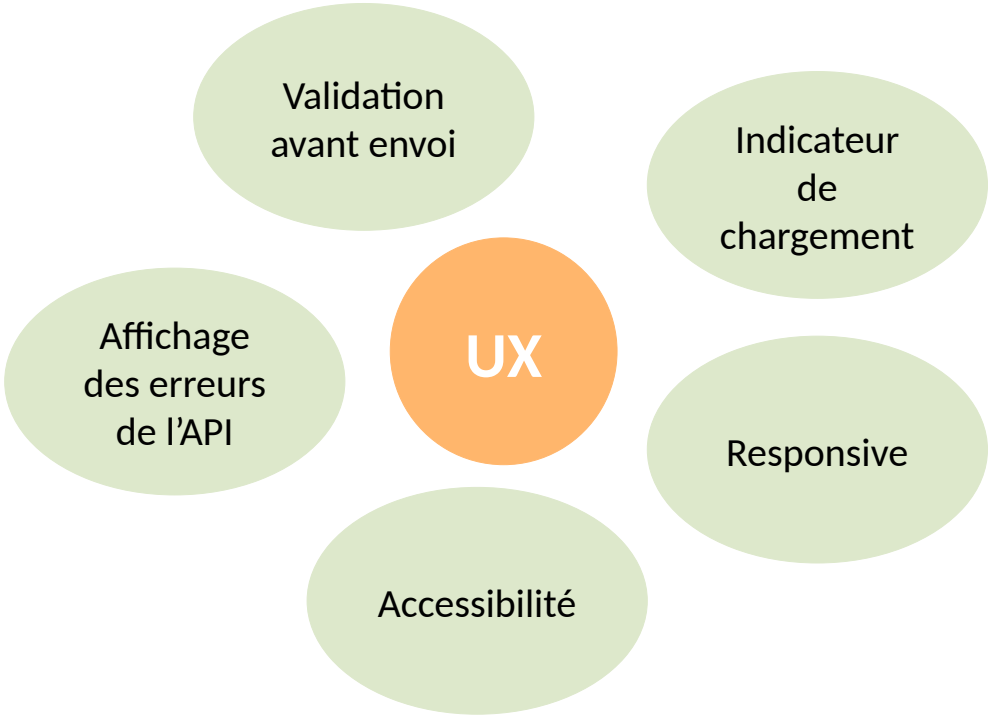
```
function App() {
  return (
    <Routes>
        <Route path="/login" element={<LoginPage />} />

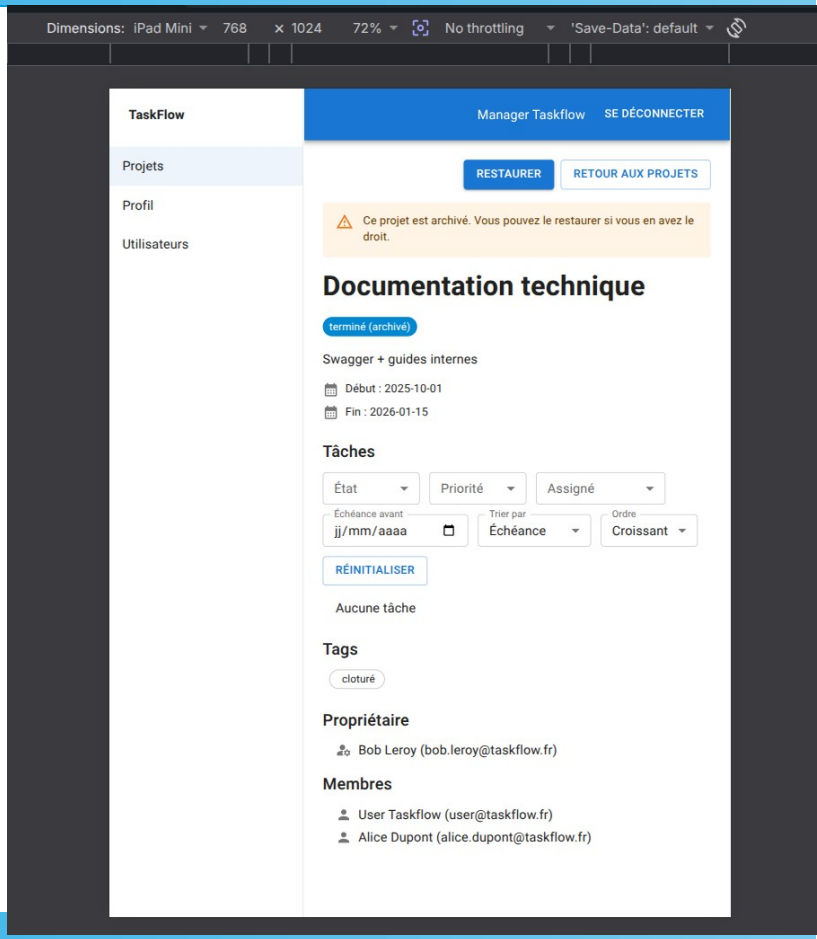
        <Route element={<ProtectedRoute />}>
            <Route element={<DashboardLayout />}>
                <Route path="/projects" element={<ProjectsPage />} />
                <Route path="/projects/create" element={<ProjectCreatePage />} />
                <Route path="/projects/:id" element={<ProjectDetailPage />} />
                <Route path="/projects/:id/edit" element={<ProjectEditPage />} />
                <Route path="/profile" element={<ProfilePage />} />
                <Route element={<ManagerRoute />}>
                    <Route path="/users" element={<UsersPage />} />
                </Route>
            </Route>
        </Route>

        <Route path="*" element={<Navigate to="/projects" />} />
    </Routes>
  )
}
```

Front-end

Expérience utilisateur :





Qualité et déploiement

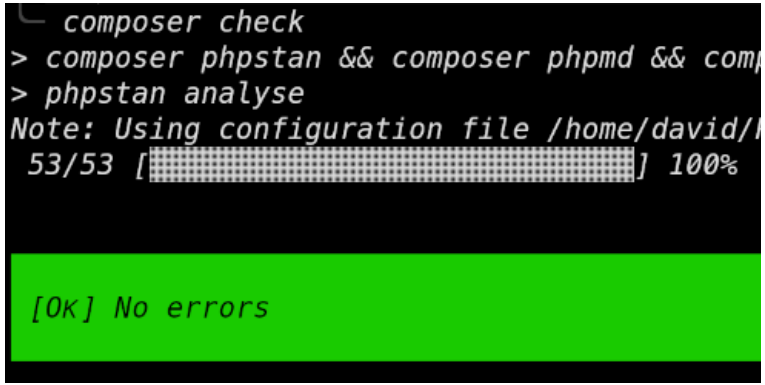
Qualité du code :

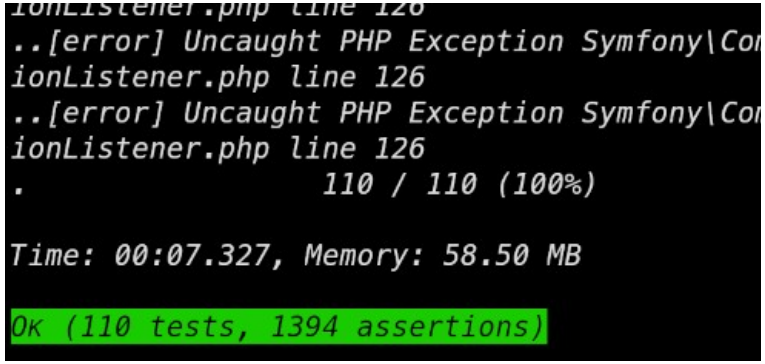
API (Symfony)	Web (React)
PHP CS Fixer : style @Symfony / PSR-12	ESLint : règles JS + React Hooks
PHPStan : analyse statique (+ extensions Symfony, Doctrine, PHPUnit	
PHPMD : code mort, complexité, nommage	

Une commande côté API, composer check :

PHPStan PHPMD

PHP CS Fixer PHPUnit





Qualité et déploiement

Stratégie de tests :



110 tests fonctionnels PHPUnit

- **Base SQLite en mémoire : rapide, isolée**
- **Cas nominaux (création, filtres…)**
- **Cas de sécurité (sans token, mauvais rôle…)**

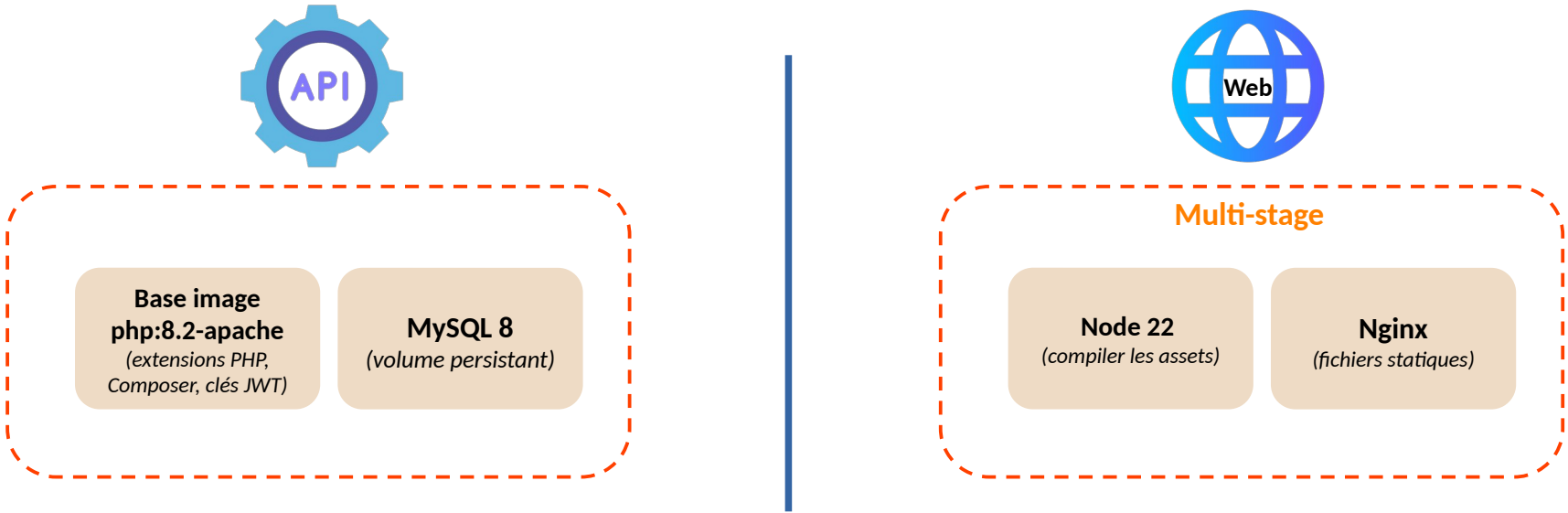


62 tests Vitest + Testing Library

- **Composants**
- **Routes protégées**
- **Permissions**

Qualité et déploiement

Conteneurisation avec Docker :



- Secrets non versionnés : .env.docker en local, variables d'environnement sur l'hébergeur
- Lancement en une commande : docker compose up --build

Qualité et déploiement

CI/CD et mise en production :



Installation des dépendances

Génération des clés JWT de test

Composer check

Build Docker



npm ci

lint

tests

Build Vite
+ Build Docker

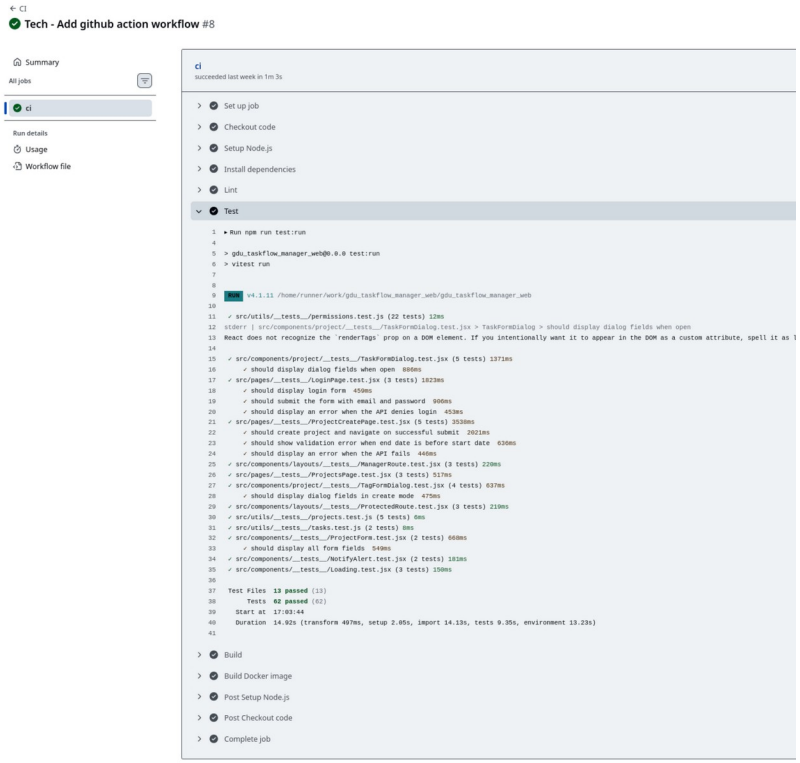


Push d'une branche

CI au vert

Pull request
+ Merge

Déploiement Render



Démo

